

1. Getting Started

1.1. Running Python Interpreter

Python comes with an interactive interpreter. When you type `python` in your shell or command prompt, the python interpreter becomes active with a `>>>` prompt and waits for your commands.

```
$ python
Python 3.7.4 (v3.7.4:e09359112e, Jul  8 2019, 14:54:52)
[Clang 6.0 (clang-600.0.57)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Now you can type any valid python expression at the prompt. python reads the typed expression, evaluates it and prints the result.

```
>>> 42
42
>>> 4 + 2
6
```

Problem 1: Open a new Python interpreter and use it to find the value of `2 + 3`.

1.2. Running Python Scripts

Open your text editor, type the following text and save it as `hello.py`.

```
print("hello, world!")
```

And run this program by calling `python hello.py`. Make sure you change to the directory where you saved the file before doing it.

```
$ python hello.py
hello, world!
```

1.3. Datatypes

Python has support for all basic datatypes and also have very powerful compound datatypes.

Python has integers.

```
>>> 1 + 2
3
```

Python is pretty good at handling very large numbers as well. For example, let us try to compute 2 raises to the power of 1000.

```
>>> 2 ** 1000
1071508607186267320948425049060001810561404811705533607443750388370351051124936122493198378815695
```



That is a pretty big numbers, isn't it? Can you count how many digits it has?

Python has floating point numbers.

```
>>> 1.2 + 2.3
3.5
```

Python has strings.

```
>>> "hello world"
'hello world'
>>> print("hello world")
hello world
```

String can be enclosed either in single quotes or double quotes. Both are exactly the same. In Python, strings are very versatile and it very easy to work with them.

```
>>> 'hello' + 'world'
'helloworld'

>>> "hello" * 3
'hellohellohello'

>>> print("=" * 40)
=====
```

The built-in function `len` is used to find the length of a string.

```
>>> len('helloworld')
10
```

Python supports multi-line strings too. They are enclosed in three double quotes or three single quotes.

```
text = """This is a multi-line string.
Line 2
Line 3
and the text may have "quotes" too.
"""
```

```
>>> print(text)
This is a multi-line string.
Line 2
Line 3
and the text may have "quotes" too.
```

Python supports the usual escape codes. `\n` indicates new line, `\t` indicates a tab etc.

```
>>> print("a\nb\nc")
a
b
c
```

Python has lists. Lists are one of the most useful data types Python.

```
>>> x = ["a", "b", "c"]
>>> x
['a', 'b', 'c']
>>> len(x)
3
>>> x[1]
'b'
```

Python has another datatype called *tuple* for representing fixed width records. Tuples behave just like lists, but they are immutable.

```
>>> point = (2, 3)
>>> point
(2, 3)
```

When writing tuples, the parenthesis can be omitted most of the times.

```
>>> point = 2, 3
>>> point
(2, 3)
```

It is also possible to assign a tuple multiple values at once:

```
>>> yellow = (255, 255, 0)
>>> r, g, b = yellow
>>> print(r, g, b)
255 255 0
```

Python has a `dictionary` datatype for representing name-value pairs.

```
>>> person = {"name": "Alice", "email": "alice@example.com"}
>>> person['name']
'Alice'
>>> person['email']
'alice@example.com'
```

Python has a `set` datatype too. A set is an unordered collection of elements.

```
>>> x = {1, 2, 3, 2, 1}
>>> x
{1, 2, 3}
```

Python has a `boolean` type. It has two special values `True` and `False` to represent truth and false.

Finally, Python has a special type called `None` to represent nothing.

```
>>> x = None
>>> print(x)
None
```

Now you know most of the common data structures of Python. While they look very simple, mastering them takes a bit of practice. Make sure you go through all the examples and the practice problems in the subsequent sections.

1.4. Variables

You've already seen variables in the previous section. Let us look at them closely now.

In Python, variables don't have a type. They are just placeholders which can hold any type of values.

```
>>> x = 5
>>> x
5
>>> x = 'hello'
>>> x
'hello'
```

It is important to notice the difference between variables and strings. Often new programmers get tricked by this. Can you spot any error in the following example?

```
name = "Alice" print("name")
```

1.5. Functions

Python has many built-in functions. The `print` is the most commonly used built-in function.

```
>>> print('hello')
hello
>>> print('hello', 1, 2, 3)
hello 1 2 3
```

We've also see the `len` function in the previous sections. The `len` function computes the length of a string, list or other collections.

```
>>> len("hello")
5
>>> len(['a', 'b', 'c'])
3
```

One important thing about Python is that it doesn't allow operations on incompatible data types. For example:

```
>>> 5 + "2"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

That is because it is not possible to add a number to a string. We need to either convert `5` into a string or `"2"` into a number. The built-in function `int` converts a string into a number and the `str` function converts any value into a string.

```
>>> int("5")
5
>>> str(5)
'5'
```

```
>>> 5 + int("2")
7
>>> str(5) + "2"
'52'
```

1.5.1. Example: Counting the number of digits in a number

Let us write a program to compute number of digits in a number. Let us look at some numbers first.

```
>>> 12345
12345
```

```
>>> 2 ** 100
1267650600228229401496703205376
```

```
>>> 2 ** 1000
1071508607186267320948425049060001810561404811705533607443750388370351051124936122493198378815695
```



We can combine the previously mentioned built-in functions to solve this.

```
>>> len(str(12345))
5
>>> len(str(2 ** 100))
31
>>> len(str(2 * 1000))
302
```

1.6. Writing Custom Functions

Just like a value can be associated with a name, a piece of logic can also be associated with a name by defining a function.

```
>>> def square(x):
...     return x * x
...
>>> square(5)
25
```

The body of the function is indented. Indentation is the Python's way of grouping statements.

The `...` is the secondary prompt, which the Python interpreter uses to denote that it is expecting some more input.

The functions can be used in any expressions.

```
>>> square(2) + square(3)
13
>>> square(square(3))
81
```

Existing functions can be used in creating new functions.

```
>>> def sum_of_squares(x, y):
...     return square(x) + square(y)
...
>>> sum_of_squares(2, 3)
13
```

Functions are just like other values, they can assigned, passed as arguments to other functions etc.

```
>>> f = square
>>> f(4)
16

>>> def fxy(f, x, y):
...     return f(x) + f(y)
...
>>> fxy(square, 2, 3)
13
```

It is important to understand, the scope of the variables used in functions.

Lets look at an example.

```
x = 0
y = 0
def incr(x):
    y = x + 1
    return y
incr(5)
print(x, y)
```

Variables assigned in a function, including the arguments are called the local variables to the function. The variables defined in the top-level are called global variables.

Changing the values of `x` and `y` inside the function `incr` won't effect the values of global `x` and `y`.

But, we can use the values of the global variables.

```
pi = 3.14
def area(r):
    return pi * r * r
```

When Python sees use of a variable not defined locally, it tries to find a global variable with that name.

However, you have to explicitly declare a variable as `global` to modify it.

```
numcalls = 0
def square(x):
    global numcalls
    numcalls = numcalls + 1
    return x * x
```

Problem 2: How many multiplications are performed when each of the following lines of code is executed?

```
print(square(5))
print(square(2*5))
```

Problem 3: What will be the output of the following program?

```
x = 1
def f():
    return x
print(x)
print(f())
```

Problem 4: What will be the output of the following program?


```
x = 1
def f():
    x = 2
    return x
print(x)
print(f())
print(x)
```

Problem 5: What will be the output of the following program?

```
x = 1
def f():
    y = x
    x = 2
    return x + y
print(x)
print(f())
print(x)
```

Problem 6: What will be the output of the following program?

```
x = 2
def f(a):
    x = a * a
    return x
y = f(3)
print(x, y)
```

Functions can be called with keyword arguments.

```
>>> def difference(x, y):
...     return x - y
...
>>> difference(5, 2)
3
>>> difference(x=5, y=2)
3
>>> difference(5, y=2)
3
>>> difference(y=2, x=5)
3
```

And some arguments can have default values.

```
>>> def increment(x, amount=1):
...     return x + amount
...
>>> increment(10)
11
>>> increment(10, 5)
15
>>> increment(10, amount=2)
12
```

There is another way of creating functions, using the `lambda` operator.

```
>>> cube = lambda x: x ** 3
>>> fxy(cube, 2, 3)
35
>>> fxy(lambda x: x ** 3, 2, 3)
35
```

Notice that unlike function definition, lambda doesn't need a `return`. The body of the `lambda` is a single expression.

The `lambda` operator becomes handy when writing small functions to be passed as arguments etc. We'll see more of it as we get into solving more serious problems.

Python provides some useful built-in functions.

```
>>> min(2, 3)
2
>>> max(3, 4)
4
```

The built-in function `len` computes length of a string.

```
>>> len("helloworld")
10
```

The built-in function `int` converts string to integer and built-in function `str` converts integers and other type of objects to strings.

```
>>> int("50")
50
>>> str(123)
"123"
```

Problem 7: Write a function `count_digits` to find number of digits in the given number.

```
>>> count_digits(5)
1
>>> count_digits(12345)
5
```

Methods are special kind of functions that work on an object.

For example, `upper` is a method available on string objects.

```
>>> x = "hello"
>>> print(x.upper())
HELLO
```

As already mentioned, methods are also functions. They can be assigned to other variables and can be called separately.

```
>>> f = x.upper
>>> f()
'HELLO'
```

Problem 8: Write a function `istrcmp` to compare two strings, ignoring the case.

```
>>> istrcmp('python', 'Python')
True
>>> istrcmp('LaTeX', 'Latex')
True
>>> istrcmp('a', 'b')
False
```

1.7. Conditional Expressions

Python provides various operators for comparing values. The result of a comparison is a boolean value, either `True` or `False`.

```
>>> 2 < 3
True
>>> 2 > 3
False
```

Here is the list of available conditional operators.

- `==` equal to
- `!=` not equal to
- `<` less than
- `>` greater than
- `<=` less than or equal to
- `>=` greater than or equal to

It is even possible to combine these operators.

```
>>> x = 5
>>> 2 < x < 10
True
>>> 2 < 3 < 4 < 5 < 6
True
```

The conditional operators work even on strings - the ordering being the lexical order.

```
>>> "python" > "perl"
True
>>> "python" > "java"
True
```

There are few logical operators to combine boolean values.

- `a and b` is `True` only if both `a` and `b` are `True`.
- `a or b` is `True` if either `a` or `b` is `True`.
- `not a` is `True` only if `a` is `False`.

```
>>> True and True
True
>>> True and False
False
>>> 2 < 3 and 5 < 4
False
>>> 2 < 3 or 5 < 4
True
```

Problem 9: What will be output of the following program?

```
print(2 < 3 and 3 > 1)
print(2 < 3 or 3 > 1)
print(2 < 3 or not 3 > 1)
print(2 < 3 and not 3 > 1)
```

Problem 10: What will be output of the following program?

```
x = 4
y = 5
p = x < y or x < z
print(p)
```

The `if` statement is used to execute a piece of code only when a boolean expression is true.

```
>>> x = 42
>>> if x % 2 == 0: print('even')
even
>>>
```

In this example, `print('even')` is executed only when `x % 2 == 0` is `True`.

The code associated with `if` can be written as a separate indented block of code, which is often the case when there is more than one statement to be executed.

```
>>> if x % 2 == 0:
...     print('even')
...
even
>>>
```

The `if` statement can have optional `else` clause, which is executed when the boolean expression is `False`.

```
>>> x = 3
>>> if x % 2 == 0:
...     print('even')
... else:
...     print('odd')
...
odd
>>>
```

The `if` statement can have optional `elif` clauses when there are more conditions to be checked. The `elif` keyword is short for `else if`, and is useful to avoid excessive indentation.

```
>>> x = 42
>>> if x < 10:
...     print('one digit number')
... elif x < 100:
...     print('two digit number')
... else:
...     print('big number')
...
two digit number
>>>
```

Problem 11: What happens when the following code is executed? Will it give any error? Explain the reasons.

```
x = 2
if x == 2:
    print(x)
else:
    print(y)
```

Problem 12: What happens the following code is executed? Will it give any error? Explain the reasons.

```
x = 2
if x == 2:
    print(x)
else:
    x +
```

1.8. Lists

Lists are one of the great datastructures in Python. We are going to learn a little bit about lists now. Basic knowledge of lists is required to be able to solve some problems that we want to solve in this chapter.

Here is a list of numbers.

```
>>> x = [1, 2, 3]
```

And here is a list of strings.

```
>>> x = ["hello", "world"]
```

List can be heterogeneous. Here is a list containing integers, strings and another list.

```
>>> x = [1, 2, "hello", "world", ["another", "list"]]
```

The built-in function `len` works for lists as well.

```
>>> x = [1, 2, 3]
>>> len(x)
3
```

The `[]` operator is used to access individual elements of a list.

```
>>> x = [1, 2, 3]
>>> x[1]
2
>>> x[1] = 4
>>> x[1]
4
```

The first element is indexed with `0`, second with `1` and so on.

We'll learn more about lists in the next chapter.

1.9. Modules

Modules are libraries in Python. Python ships with many standard library modules.

A module can be imported using the `import` statement.

Lets look at `time` module for example:

```
>>> import time
>>> time.asctime()
'Tue Sep 11 21:42:06 2012'
```

The `asctime` function from the `time` module returns the current time of the system as a string.

The `sys` module provides access to the list of arguments passed to the program, among the other things.

The `sys.argv` variable contains the list of arguments passed to the program. As a convention, the first element of that list is the name of the program.

Lets look at the following program `echo.py` that prints the first argument passed to it.

```
import sys
print(sys.argv[1])
```

Lets try running it.

```
$ python echo.py hello
hello
$ python echo.py hello world
hello
```

There are many more interesting modules in the standard library. We'll learn more about them in the coming chapters.

Problem 13: Write a program `add.py` that takes 2 numbers as command line arguments and prints its sum.

```
$ python add.py 3 5
8
$ python add.py 2 9
11
```